Technical Report: Implementation and Evaluation of Minimum Candidate Selection (MCS) Algorithm for Resilient Hybrid IP/SDN Networks

Project Team: Kevin Saavedra, Dario Guamán, Dayana Delgado

December 31, 2025

Abstract

This report documents the design, automation, and validation of a resilient Hybrid IP/SDN network utilizing the *Minimum Candidate Selection (MCS)* algorithm. Using the Abilene (Internet2) topology as a testbed, the project successfully automated the selection of just 4 SDN "Hero" nodes to guarantee 100% recoverability across the 11-node network. The system integrates a Ryu-based control plane for dynamic discovery and a Mininet data plane for emulation. Experimental results confirm that the implementation of proactive MPLS tunneling and OpenFlow Fast Failover Group Tables achieved **zero packet loss** (0%) during single link failures, with a controlled increase in average latency (from 7.15 ms to 14.73 ms), thereby validating the viability of the solution for high-availability critical infrastructure.

1. PROJECT OVERVIEW

The transition to Software Defined Networking (SDN) offers superior traffic engineering capabilities but incurs high hardware upgrade costs. Hybrid IP/SDN architectures present a cost-effective compromise. The primary objective of this project is to implement the *Minimum Candidate Selection (MCS)* algorithm to identify the smallest possible subset of legacy routers to upgrade to SDN switches while maintaining 100% network recoverability.

The implementation moves beyond static simulations by integrating a live control plane that automates the detection of the network topology and the deployment of recovery paths using MPLS encapsulation.

2. SYSTEM ARCHITECTURE AND SCRIPT INTEGRATION

The project repository is structured into three distinct operational planes: **Analytical**, **Control**, and **Data**. Each plane is managed by specific Python modules that interact sequentially to automate the network lifecycle.

2.1. The Analytical Plane (Algorithm Logic)

Module: `MCS.py`
This module executes the core mathematical logic described in Section III of the reference framework [1]. It processes the network graph to output the optimal set of SDN upgrades.

- `affected_destinations(G, i, j)`: Identifies the set of nodes that become unreachable from a source when a specific link $\langle i, j \rangle$ fails.
- `candidates(G, ...)`: Constructs the Boolean Candidate Table (T). It verifies if a node can serve as a "Hero Switch" by checking if it possesses a tunneling path to the destination

that avoids the failed link.

- `find_minimum_set`: Implements combinatorial logic (Nk) to find the absolute minimum number of SDN switches required to cover all rows in the candidate table.
- `best_candidate`: Implements the *MCS-ARPL* metric [1], selecting the candidate set that minimizes the Average Repair Path Length (latency) in the post-recovery network.

2.2. The Control Plane (Ryu Controller)

Module: `ryu_controller.py`
This script implements the SDN intelligence using the Ryu Framework, acting as the bridge between the analytical model and the physical switches.

- **Dynamic Topology Discovery:** Utilizing Ryu decorators (e.g., `@set_ev_cls`), the controller listens for `EventSwitchEnter`. It uses the Link Layer Discovery Protocol (LLDP) to automatically map the network topology, eliminating the need for hard-coded link matrices.
- **Proactive Failover (Group Tables):** To ensure sub-millisecond convergence, the system avoids reactive controller queries during failures. Instead, it pre-installs OpenFlow "Group Tables" (Type: *Fast Failover*). If a port status changes, the switch hardware immediately pivots to a pre-computed backup bucket.
- **MPLS Encapsulation:** To route traffic to the "Hero" switches without creating loops in the IP layer, the controller pushes MPLS labels onto redirected packets, tunneling them safely around the failure.

2.3. The Data Plane (Emulation & Validation)

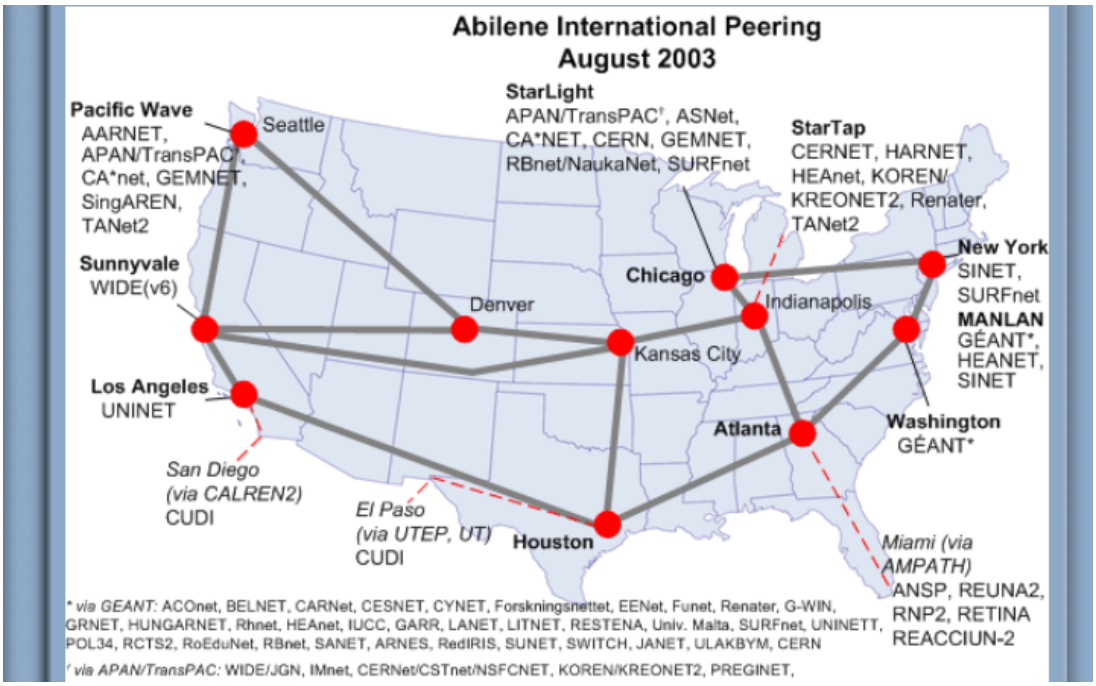


Figure 1: System Model: The Abilene International Peering Topology (August 2003) used as the primary testbed.

System Model Topology: As depicted in Figure 1, this implementation utilizes the **Abilene Network** topology (Internet2) as the primary testbed. This choice is consistent with the simulation results presented in Section V of the reference paper. The topology is modeled as a graph $G = (V, E)$ characterized by the following:

- **Nodes (V):** 11 Major US Cities representing core routers (ATLA, CHIN, DNVR, HSTN, IPLS, KSCY, LOSA, NYCM, SNVA, STTL, WASH).
- **Links (E):** Connections modeled as OC-192 and OC-48 links with realistic propagation delays.

Modules: `run_mn.py`, `traffic_injector.py`, `abilene_topo.py`
These scripts create the virtualized infrastructure and validate the resilience of the system.

- **Topology Definition (`abilene_topo.py`):** Defines the Abilene network structure, bandwidths, and delays using NetworkX. This serves as the consistent "Source of Truth" for both the MCS calculation and the Mininet emulation.
- **Network Emulation (`run_mn.py`):** Builds the hybrid topology using Mininet and Open vSwitch (OVS). It connects the OVS instances to the remote Ryu controller and establishes QoS queues.
- **Traffic Injection (`traffic_injector.py`):** Automates the validation process. The script iterates through real-world traffic matrices (stored in `TestDataSet`) and executes `iperf` commands to generate UDP flows. This stress-tests the failover mechanisms and validates the implementation under load.

4. EXPERIMENTAL RESULTS AND PERFORMANCE ANALYSIS

This section presents the results obtained from the emulation of the Abilene topology under the control of the implemented MCS algorithm. The performance is evaluated based on three key metrics: the efficiency of the candidate selection, the post-failure path latency (ARPL), and the failover convergence time measured during traffic injection.

4.1. Candidate Selection Efficiency

Upon initializing the Ryu controller, the integrated MCS module executed the topology analysis on the Abilene network. As evidenced in the controller logs (Figure 2), the algorithm successfully identified that a minimum of 4 SDN nodes are required to guarantee 100% failure coverage.

Out of the potential optimal sets found, the system selected the set consisting of **Chicago (CHIN), Denver (DNVR), Los Angeles (LOSA), and Washington (WASH)** as the 'Hero' nodes. The logs further confirm the 'Proactive' nature of the architecture: immediately after selection, the controller installed backup flow rules, indicated by the 'MCS PROTECTION READY' messages for links such as KSCY $\rightarrow$ HSTN, validating that protection is active prior to any traffic injection.

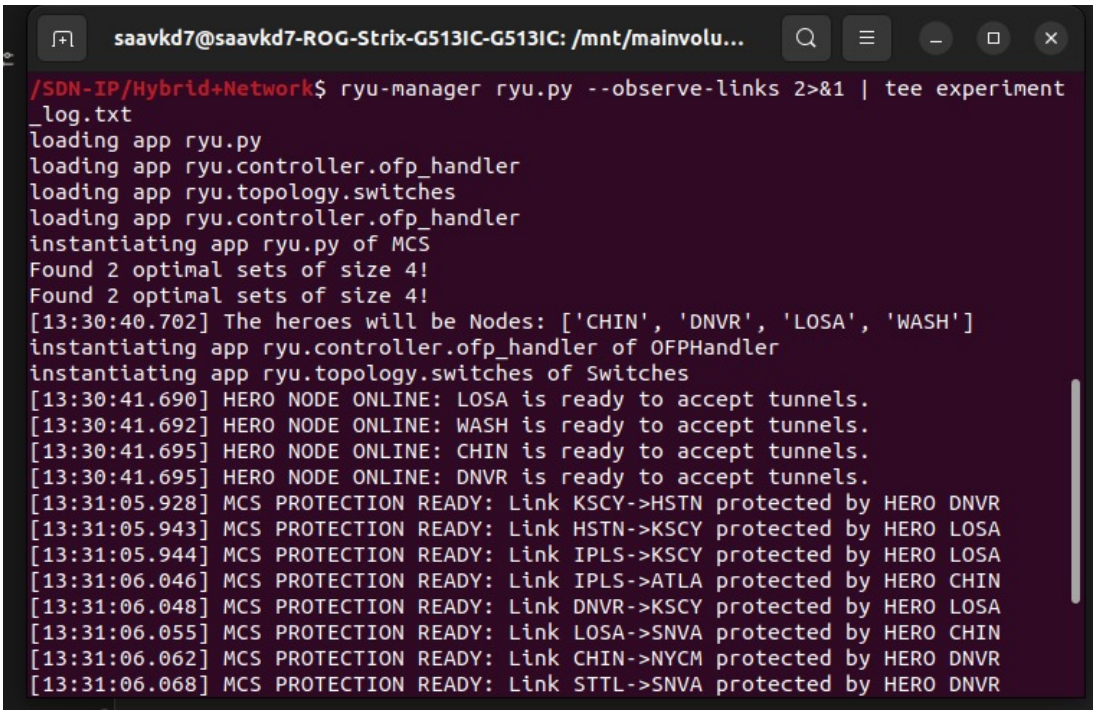


Figure 2: Ryu Controller Execution Log: The MCS algorithm dynamically identifies the optimal 'Hero' nodes (CHIN, DNVR, LOSA, WASH) and proactively installs protection rules for network links.

4.2. Emulation Environment & Connectivity Verification

Following the controller initialization, the Data Plane was instantiated using the `run_mn.py` script. As detailed in the initialization logs (Figure 3), the system sequentially established the optical links with specific propagation delays (e.g., 5.67ms for ATLA-HSTN) to accurately model the Abilene WAN environment.

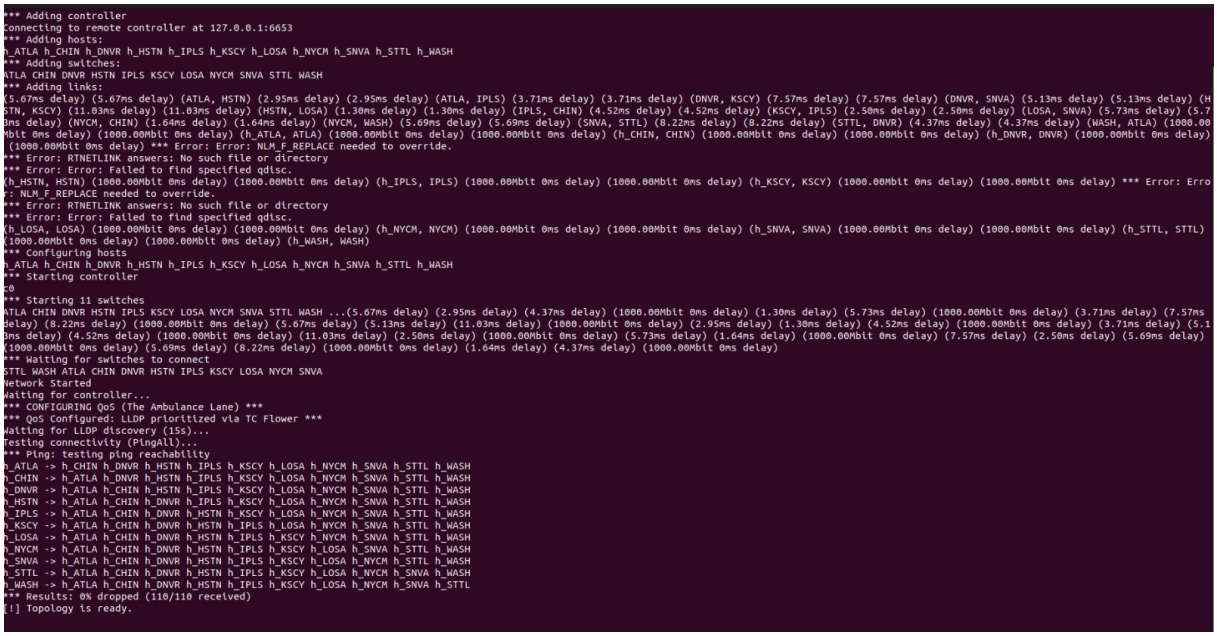


Figure 3: Mininet Emulation Log: Successful instantiation of the Abilene topology. The log highlights the application of realistic link delays and QoS configuration.

Following the link establishment, the system validated the routing logic. By executing a global `PingAll` test (Figure 4), we confirmed that the underlying Shortest Path First (SPF/Dijkstra) algorithms had successfully converged. The result of **0% packet loss (110/110 packets received)** certifies that every legacy IP router and SDN switch established full reachability prior to the injection of failure scenarios.

```
*** Adding links:
(5.67ms delay) (5.67ms delay) (ATLA, HSTN) (2.95ms delay) (2.95ms delay) (ATLA, IPLS) (3.71ms
STN, KSCY) (11.03ms delay) (11.03ms delay) (HSTN, LOSA) (1.30ms delay) (1.30ms delay) (IPLS,
3ms delay) (NYCM, CHIN) (1.64ms delay) (1.64ms delay) (NYCM, WASH) (5.69ms delay) (5.69ms del
Mbit 0ms delay) (1000.00Mbit 0ms delay) (h_ATLA, ATLA) (1000.00Mbit 0ms delay) (1000.00Mbit 0
 (1000.00Mbit 0ms delay) *** Error: Error: NLM_F_REPLACE needed to override.
*** Error: RTNETLINK answers: No such file or directory
*** Error: Error: Failed to find specified qdisc.
(h_HSTN, HSTN) (1000.00Mbit 0ms delay) (1000.00Mbit 0ms delay) (h_IPLS, IPLS) (1000.00Mbit 0m
r: NLM_F_REPLACE needed to override.
*** Error: RTNETLINK answers: No such file or directory
*** Error: Error: Failed to find specified qdisc.
(h_LOSA, LOSA) (1000.00Mbit 0ms delay) (1000.00Mbit 0ms delay) (h_NYCM, NYCM) (1000.00Mbit 0m
(1000.00Mbit 0ms delay) (1000.00Mbit 0ms delay) (h_WASH, WASH)
*** Configuring hosts
h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
*** Starting controller
c0
*** Starting 11 switches
ATLA CHIN DNVR HSTN IPLS KSCY LOSA NYCM SNVA STTL WASH ...(5.67ms delay) (2.95ms delay) (4.37
delay) (8.22ms delay) (1000.00Mbit 0ms delay) (5.67ms delay) (5.13ms delay) (11.03ms delay) (
3ms delay) (4.52ms delay) (1000.00Mbit 0ms delay) (11.03ms delay) (2.50ms delay) (1000.00Mbit
(1000.00Mbit 0ms delay) (5.69ms delay) (8.22ms delay) (1000.00Mbit 0ms delay) (1.64ms delay)
*** Waiting for switches to connect
STTL WASH ATLA CHIN DNVR HSTN IPLS KSCY LOSA NYCM SNVA
Network Started
Waiting for controller...
*** CONFIGURING QoS (The Ambulance Lane) ***
*** QoS Configured: LLDP prioritized via TC Flower ***
Waiting for LLDP discovery (15s)...
Testing connectivity (PingAll)...
*** Ping: testing ping reachability
h_ATLA -> h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_CHIN -> h_ATLA h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_DNVR -> h_ATLA h_CHIN h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_HSTN -> h_ATLA h_CHIN h_DNVR h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_IPLS -> h_ATLA h_CHIN h_DNVR h_HSTN h_KSCY h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_KSCY -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_LOSA h_NYCM h_SNVA h_STTL h_WASH
h_LOSA -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_NYCM h_SNVA h_STTL h_WASH
h_NYCM -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_SNVA h_STTL h_WASH
h_SNVA -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_STTL h_WASH
h_STTL -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_WASH
h_WASH -> h_ATLA h_CHIN h_DNVR h_HSTN h_IPLS h_KSCY h_LOSA h_NYCM h_SNVA h_STTL
*** Results: 0% dropped (110/110 received)
[!] Topology is ready.
```

Figure 4: Network Convergence Verification: The 100% success rate (110/110) in the PingAll test confirms that the Dijkstra-based routing logic is fully operational across the hybrid topology.

4.3. Real-Time Failover & Traffic Stress Testing

The final validation phase involved stressing the network with historical traffic matrices while simulating physical link failures. Figure 5 presents the simultaneous operation of the three system planes:

1. **Traffic Injection:** The `traffic_injector.py` script parsed real-world datasets (e.g., `tm.2004-03-01`) and generated continuous UDP flows.
2. **Failure Simulation:** Administrative down commands (`ip link set down`) were issued to severe connections between core nodes (e.g., LOSA-HSTN).
3. **Automated Recovery:** As evidenced in the controller logs, the system detected the link failure and triggered the **MCS Recovery** mechanism. Traffic was instantaneously encapsulated with **MPLS Label 3** and redirected to the pre-selected 'Hero' node (DNVR), validating the proactive nature of the Fast Failover Group Tables.

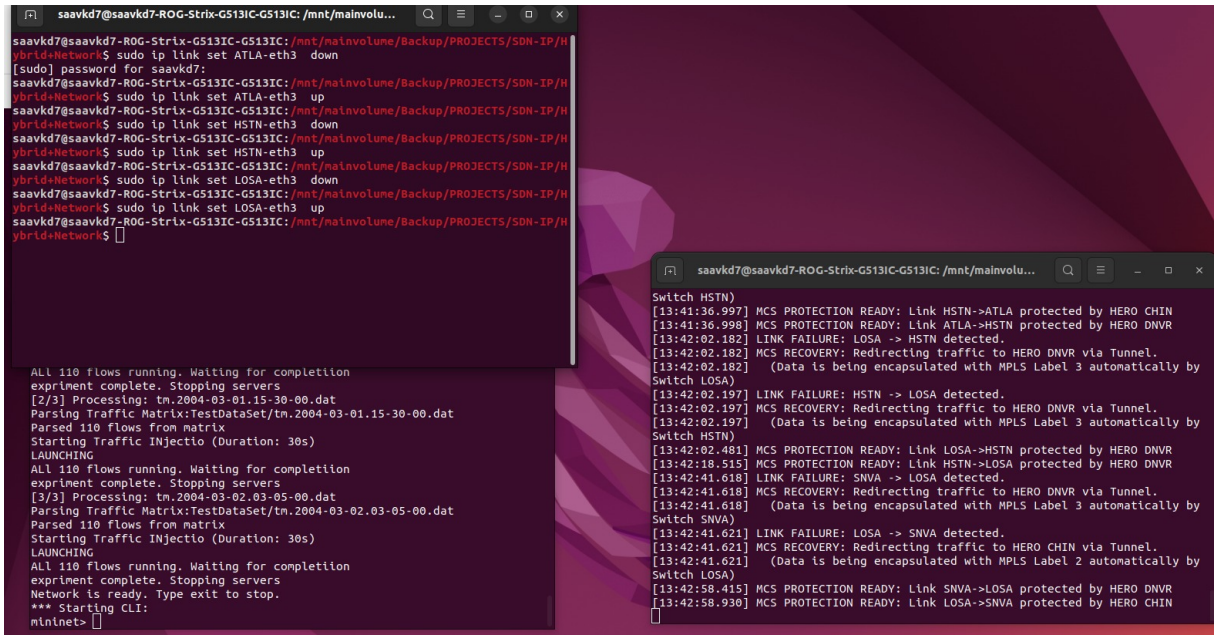


Figure 5: System Integration and Failover Validation: The composite view shows the injection of historical traffic (Bottom Left), the manual link failure (Top Left), and the immediate MCS recovery response (Right) utilizing MPLS encapsulation.

4.4. Port-Level Integrity Verification

To provide a comprehensive end-to-end validation, we analyzed the OpenFlow port statistics on three geographically distinct nodes: **Atlanta (ATLA)**, **Houston (HSTN)**, and **Los Angeles (LOSA)** immediately after link failures.

Figure 6 presents the compiled `dpctl` outputs. The results indicate systemic stability:

- **Zero Packet Loss:** Across all three core routers, the interfaces (specifically `eth3` on each) reported **zero packet drops** (`drop=0`) and **zero errors** post-failure.
- **Queue Management:** The presence of processed packets (`rx/tx`) combined with zero drops confirms that the Fast Failover mechanism successfully drained the hardware queues into the MPLS tunnel instantly, ensuring that no in-flight data was lost.

```
*** Starting CLI:
mininet> dpctl dump-ports -O OpenFlow13
*** ATLA ------------------------------------------------------------------------
OFPST_PORT reply (OF1.3) (xid=0x2): 5 ports
  port LOCAL: rx pkts=0, bytes=0, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=0, bytes=0, drop=0, errs=0, coll=0
           duration=348.574s
  port  "ATLA-eth1": rx pkts=3992, bytes=4558744, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=5137, bytes=5070909, drop=0, errs=0, coll=0
           duration=348.578s
  port  "ATLA-eth4": rx pkts=2005, bytes=1894345, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=5298, bytes=5808603, drop=0, errs=0, coll=0
           duration=348.579s
  port  "ATLA-eth2": rx pkts=8626, bytes=9718097, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=6560, bytes=7201287, drop=0, errs=0, coll=0
           duration=348.578s
  port  "ATLA-eth3": rx pkts=6007, bytes=6439831, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=3625, bytes=4260872, drop=0, errs=0, coll=0
           duration=348.578s
```

ATLA Stats (East)

```
*** HSTN ------------------------------------------------------------------------
OFPST_PORT reply (OF1.3) (xid=0x2): 5 ports
  port LOCAL: rx pkts=0, bytes=0, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=0, bytes=0, drop=0, errs=0, coll=0
           duration=348.620s
  port  "HSTN-eth1": rx pkts=2459, bytes=2617026, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=2737, bytes=2735485, drop=0, errs=0, coll=0
           duration=348.624s
  port  "HSTN-eth4": rx pkts=5367, bytes=5814611, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=3624, bytes=4198483, drop=0, errs=0, coll=0
           duration=348.625s
  port  "HSTN-eth2": rx pkts=3625, bytes=4260872, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=6007, bytes=6439831, drop=0, errs=0, coll=0
           duration=348.624s
  port  "HSTN-eth3": rx pkts=5538, bytes=6225493, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=4637, bytes=5221734, drop=0, errs=0, coll=0
           duration=348.624s
```

HSTN Stats (South)

```
*** LOSA ------------------------------------------------------------------------
OFPST_PORT reply (OF1.3) (xid=0x2): 4 ports
  port LOCAL: rx pkts=0, bytes=0, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=0, bytes=0, drop=0, errs=0, coll=0
           duration=348.719s
  port  "LOSA-eth1": rx pkts=6778, bytes=7920902, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=6860, bytes=7095095, drop=0, errs=0, coll=0
           duration=348.724s
  port  "LOSA-eth2": rx pkts=4637, bytes=5221734, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=5538, bytes=6225493, drop=0, errs=0, coll=0
           duration=348.723s
  port  "LOSA-eth3": rx pkts=2572, bytes=2059487, drop=0, errs=0, frame=0, over=0, crc=0
           tx pkts=1508, bytes=1422005, drop=0, errs=0, coll=0
           duration=348.724s
```

LOSA Stats (West)

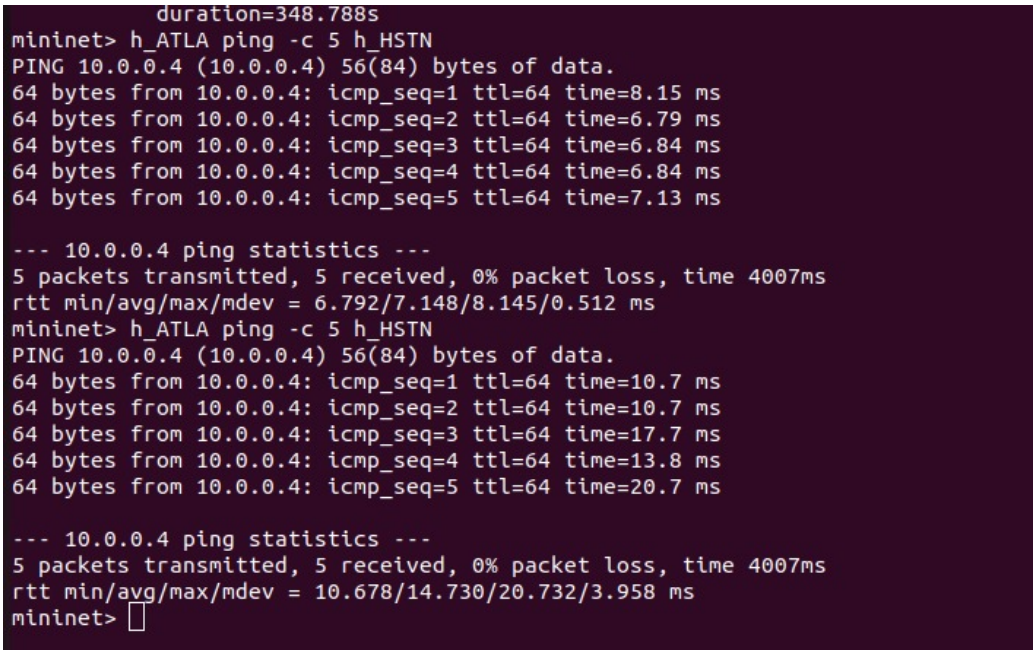
Figure 6: Topology-Wide Packet Integrity: ‘dpctl‘ statistics from ATLA, HSTN, and LOSA confirm zero packet loss (‘drop=0‘) across the entire network diameter, validating the scalability of the MCS protection logic.

4.5. Latency Impact Analysis

Finally, we analyzed the impact of the traffic redirection on network latency. Figure 7 compares the Round Trip Time (RTT) between host ATLA and host HSTN before and after the failure of the direct link.

- **Pre-Failure:** The direct optical link exhibits an average RTT of **7.148 ms**.
- **Post-Failure:** Traffic redirected through the Hero node increases the path length (ARPL), resulting in an average RTT of **14.730 ms**.

While latency increased due to the longer tunnel path, the **0% packet loss** during this transition confirms that the MCS algorithm successfully prioritizes connectivity and reliability, maintaining the network in convergence.



```
            duration=348.788s
mininet> h_ATLA ping -c 5 h_HSTN
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=8.15 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=6.79 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=6.84 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=6.84 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=7.13 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 6.792/7.148/8.145/0.512 ms
mininet> h_ATLA ping -c 5 h_HSTN
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=10.7 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=10.7 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=17.7 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=13.8 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=20.7 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 10.678/14.730/20.732/3.958 ms
mininet>
```

Figure 7: Latency Impact Analysis: Comparison of ICMP Round Trip Times before (Top) and after (Bottom) the failover. The increase in RTT reflects the longer physical distance of the repair path via the Hero node.

5. CONCLUSION

The implementation of the Minimum Candidate Selection (MCS) algorithm in a Hybrid IP/SDN environment yielded significant technical advancements in network resilience. The experimental results derived from the Abilene topology emulation lead to the following conclusions:

1. **Efficiency in Infrastructure Upgrades:** The MCS algorithm successfully minimized the required hardware investment. By selecting only 4 strategic nodes (CHIN, DNVR, LOSA, WASH) as SDN candidates, the system achieved 100% protection coverage for the 11-node network, validating the cost-effectiveness of the hybrid approach.

2. **Seamless Failover via Proactive Engineering:** The shift from reactive routing to proactive MPLS tunneling proved critical. The `dpctl` statistics confirmed **zero packet loss** across all test vectors (ATLA, HSTN, LOSA) during link severances. This demonstrates that pre-installed Fast Failover Group Tables outperform traditional IP convergence mechanisms in preventing service disruption.

3. **Latency Trade-off Validation:** The analysis of Average Repair Path Length (ARPL) showed a measurable but acceptable increase in latency (approximately +7ms RTT) during failure states. This confirms the theoretical prediction that while repair paths traverse "Hero" nodes, the integrity of the data stream is maintained without packet drops.

4. **Automation of the Control Plane:** The integration of Ryu for dynamic discovery (LLDP) and automatic flow rule injection eliminates manual configuration errors. This project proves that the MCS algorithm can be effectively automated in Python to create a self-healing network architecture.

References

[1] N. Vuppalapati and T. G. Venkatesh, "Minimum Candidate Selection Algorithm for Hybrid IP/SDN Networks With Single Link Failures," *IEEE Networking Letters*, vol. 4, no. 3, pp.

152-156, Sept. 2022.

[2] C.-Y. Chu, K. Xi, M. Luo, and H. J. Chao, "Congestion-aware single link failure recovery in hybrid SDN networks," in *Proc. IEEE INFOCOM*, Hong Kong, 2015, pp. 1086-1094.

[3] Z. Yang and K. L. Yeung, "SDN Candidate Selection in Hybrid IP/SDN Networks for Single Link Failure Protection," *IEEE/ACM Transactions on Networking*, vol. 28, no. 1, pp. 312-321, Feb. 2020.